

AI Platform Foundation Proposal (Pre-Implementation)

Status: Proposal for discussion before implementation

Project: EduLatics Backend

Goal: Design a scalable AI architecture that supports every future AI feature without requiring major rewrites.

Current Project Status

The backend has reached a very strong foundation.

Core Infrastructure

- ☒ Authentication (JWT + Refresh Tokens)
- ☒ RBAC & Permissions
- ☒ Multi-Tenancy
- ☒ Audit Logs
- ☒ Background Scheduler
- ☒ Escalation Engine
- ☒ CI/CD Pipeline
- ☒ 47/47 Tests Passing

Core School Modules

- ☒ Schools
- ☒ Users
- ☒ Students
- ☒ Classes
- ☒ Attendance
- ☒ Results
- ☒ Fees

At this point, the ERP foundation is stable enough to begin AI development.

Question

Should we immediately begin implementing the AI platform (Providers, Prompts, Conversations, etc.)?

Recommendation: No.

Instead, first design the complete AI product.

The architecture should be driven by real AI features—not assumptions.

Why This Matters

If we immediately create tables like:

- AIProvider
- AIPrompt
- AIConversation

we are making assumptions about future requirements.

Example:

Do we actually need conversations?

Maybe:

- Principal AI does not.
- Teacher AI does not.
- Parent AI does not.
- Student AI does.

If conversations are only needed by one AI product, then designing conversation infrastructure before understanding the product creates unnecessary complexity.

Recommended Approach

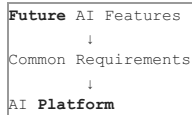
Instead of designing the AI platform first:

AI Platform

↓

Future Features

Reverse it.



This leads to cleaner architecture and fewer future redesigns.

Step 1 — Define Every AI Product

Before writing AI code, list every AI capability EduLatics will eventually provide.

1. Principal AI

Possible capabilities:

- Attendance insights
- Dropout risk detection
- Fee recovery suggestions
- Teacher performance summaries
- School health reports
- Monthly operational reports
- Student trend analysis
- AI-generated executive summaries

Needs:

- Analytics
- Reports
- School-wide data
- No long conversations required

2. Teacher AI

Possible capabilities:

- Homework generation
- Quiz generation
- Lesson planning
- Worksheet generation
- Weak student detection
- Classroom recommendations
- Exam paper generation

Needs:

- Prompt templates
- Subject context
- Curriculum context

May not require conversations.

3. Parent AI

Possible capabilities:

- Explain report cards
- Explain attendance
- Homework assistance
- Fee explanations
- Student progress summaries

Needs:

- Student-specific permissions
- Personalized context

Possibly lightweight conversations.

4. Student AI

Possible capabilities:

- Tutor
- Explain concepts
- Practice questions
- Study planner
- Personalized learning
- Revision assistant

Needs:

- Long conversations
- Memory

- Chat history

This is likely the first feature that truly needs conversation tables.

5. Admin AI

Possible capabilities:

- Admission forecasting
- Revenue analysis
- Fee projections
- Resource planning
- Staffing insights

Needs:

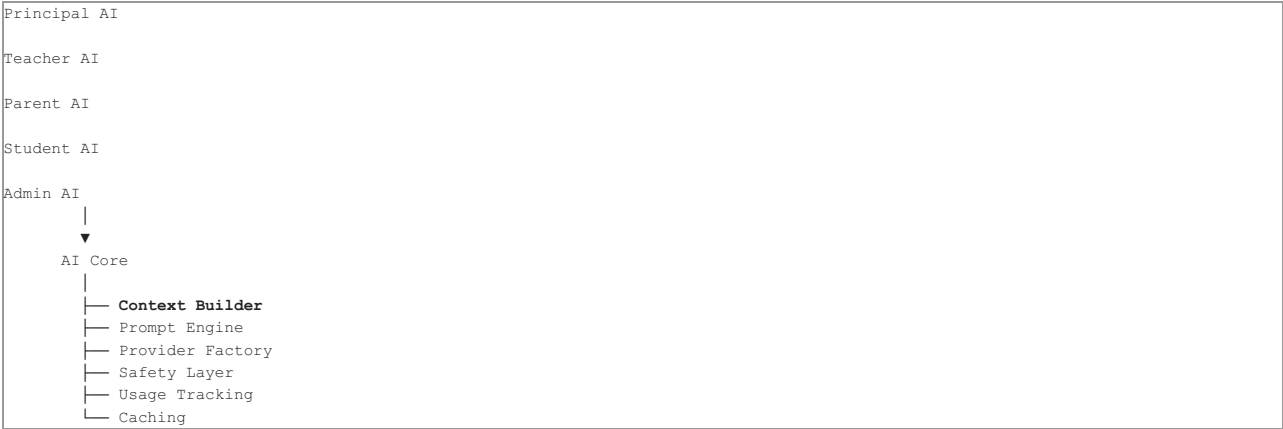
- Analytics
- Reports

No conversations required.

Step 2 — Extract Common Components

Once every AI feature is listed, identify what they share.

Example:



This ensures the AI platform is built from real requirements.

Recommended Service Architecture

Instead of:



Use:



Benefits:

- Small services
- Easier testing
- Easier maintenance
- Easier permissions
- Easier scaling

Instead of one enormous `AIService`, each AI product owns its business logic while sharing the common AI infrastructure.

Recommended Database Roadmap

Do **not** build every AI table immediately.

Instead, introduce them in phases.

Phase A

Only build:

```
AIProvider
```

```
AIUsage
```

Purpose:

- Select LLM provider
- Track token usage
- Track cost

Enough to support the first AI features.

Phase B

Add:

```
PromptTemplate
```

Only when prompt management/versioning becomes necessary.

Phase C

Add:

```
Conversation
```

```
ConversationMessage
```

Only when chat-based AI becomes a requirement.

Phase D

Add:

```
KnowledgeBase
```

```
Embedding
```

```
Document
```

```
Vector Store Metadata
```

Only when Retrieval-Augmented Generation (RAG) is introduced.

Why Delay Conversation Models?

Current AI features like:

- Attendance insights
- Report summaries
- Homework generation
- Quiz generation

do not require persistent conversations.

Building conversation infrastructure early adds:

- Extra models
- Extra APIs
- Extra migrations
- Extra testing
- Extra maintenance

without immediate value.

Recommended AI Core Responsibilities

The AI Core should remain generic.

Responsibilities:

- Provider selection
- Prompt execution
- Context assembly
- Token accounting
- Safety checks
- Retry logic
- Cost tracking
- Rate limiting
- Caching

It should **not** know anything about:

- Students
- Teachers
- Attendance
- Results
- Fees

Those remain business modules.

Context Builder

Instead of importing repositories directly:

```
StudentRepository
AttendanceRepository
FeeRepository
```

the AI Core should receive prepared data.

Example:

```
PrincipalAIService
|
|
Collect Attendance
Collect Results
Collect Fees
|
|
Context Builder
|
|
AI Core
```

This avoids circular dependencies and keeps the AI platform independent.

Provider Factory

The AI Core should never directly instantiate OpenAI or another provider.

Instead:

```
AI Core
|
|
Provider Factory
|
|
OpenAI
Gemini
Claude
DeepSeek
Local Models
```

Switching providers should require configuration changes, not code changes.

Long-Term Vision

As the platform grows, new AI products should plug into the existing AI Core without changing its architecture.

Example:

```
Library AI
Transport AI
Finance AI
HR AI
Admissions AI
```

Each product implements its own business logic while reusing the shared AI infrastructure.

Proposed Next Milestone

Before implementing any AI models or services, create an **AI Product Specification** document containing:

For every AI feature:

- Purpose
- Target user
- Inputs
- Outputs
- Required permissions
- Required school data
- Context sources
- Prompt requirements
- Conversation requirement (Yes/No)
- Estimated token usage
- Expected latency
- Future extensibility

Once this specification is complete, the AI Platform Foundation can be implemented with much greater confidence and significantly lower risk of future redesign.

Recommendation

Do not begin coding the AI infrastructure yet.

Instead:

1. Define every planned AI feature across all user roles.
2. Identify shared infrastructure requirements.
3. Finalize the AI Product Specification.
4. Design the architecture based on actual use cases.
5. Implement the AI Platform Foundation in phased iterations.

This feature-driven approach minimizes premature abstraction, keeps the architecture maintainable, and reduces the likelihood of expensive refactoring as EduLatics evolves.